Frontend and Lambda Project Report

1. Executive Summary

The CCMS portal is a serverless-first monitoring and command interface for distributed streetlight panels. The system combines:

- A Next.js 16 + React 19 frontend (operator console)
- AWS Lambda functions (API and ingestion)
- DynamoDB tables for metadata and telemetry
- AWS IoT + SQS pipeline (edge to cloud ingestion)

The frontend provides fleet visibility (dashboard, maps, analytics, alerts) and command operations (manual relay commands, RTC schedule updates, panel provisioning). Lambda functions expose authenticated CRUD and telemetry APIs, and ingest telemetry payloads from SQS into DynamoDB.

This report documents architecture, implementation details, data flow, auth model, API behavior, deployment assumptions, current risks, and recommendations.

2. Scope and Objectives

This report focuses specifically on:

- Frontend application in `ccms/`
- Lambda functions in `Lambda/`

Primary objectives:

- Describe how the frontend is structured and how each feature works
- Describe Lambda endpoint behavior, validation, and data persistence logic
- Document integration contracts between frontend and backend
- Identify implementation strengths, gaps, and production hardening needs

3. System Context

3.1 Logical Components

1. Edge meters publish telemetry to AWS IoT Core.
2. IoT rules deliver payloads (via SQS) to ingestion Lambda.
3. Ingestion Lambda writes telemetry into `MeterTelemetry` (DynamoDB).
4. API Lambda serves metadata, history, snapshot, and mutation endpoints over HTTP.
5. Frontend consumes API Lambda using key-based headers.

3.2 Data Stores

- `PanelMetadata` (DynamoDB): panel identity, status, location, firmware, and mutable configuration fields.
- `MeterTelemetry` (DynamoDB): time-series telemetry indexed by `meter_id` + `timestamp`.

4. Frontend Detailed Report

4.1 Technology Stack

- Framework: Next.js 16 (`output: "export"` static export)
- UI runtime: React 19
- Styling: Tailwind CSS v4 + HeroUI v3
- Visualization: Recharts
- Mapping: Leaflet + React Leaflet
- Date/time utils: date-fns
- Iconography: lucide-react

4.2 Application Architecture

Frontend is organized into three layers:

1. Route layer (`app/`)
    - Defines pages, user flows, and route-level state/effects.
2. Component layer (`components/`)
    - Shared shell, auth boundary, map widgets, and UI primitives.
3. API/data layer (`lib/api/`, `lib/auth/`)
    - Header auth injection, typed contracts, and backend-to-UI data shaping.

Because static export mode is enabled, the frontend is a static app that calls external APIs from the browser at runtime.

4.3 Authentication and Session Model

Auth is key-based and client-managed:

- Login captures `dashboardKey` (required) and `adminKey` (optional).
- Session is stored in localStorage (`ccms_dashboard_session`).
- API wrapper sends headers:
    - `x-dashboard-key` for all authenticated calls
    - `x-admin-key` when present (required by backend for mutating methods)

Role derivation is local:

- Admin when `adminKey` exists
- Operator when only dashboard key exists

Route protection:

- Console routes use `RequireAuth` wrapper to redirect unauthenticated users to `/login`.

4.4 Route and Feature Breakdown

4.4.1 Landing (`/`)

- Branding/entry page with shortcuts to login and dashboard.

4.4.2 Login (`/login`)

- Accepts dashboard and optional admin key.
- Stores keys via auth provider.
- Redirects authenticated users to dashboard.

4.4.3 Dashboard (`/dashboard`)

- Fetches in parallel:
  - Fleet summary (derived from snapshot)
  - Panels list
  - Active alerts
- Shows KPI cards:
  - total nodes
  - active alarms
  - cluster avg voltage
  - frequency
  - total PF
  - estimated 24h energy
- Renders fleet health bar chart (online/offline/fault).
- Renders recent alert feed.

4.4.4 Panels (`/panels`)

- Fleet inventory with:
  - status filter
  - search
  - sorting
  - pagination
  - 3 view modes: map / grid / table
- Uses dynamic map import to avoid SSR Leaflet constraints.
- Includes deep links to per-panel console page.

4.4.5 Panel Details (`/panel?id=...`)

- Core command center per node.
- Polls live every 30 seconds and supports manual refresh.
- Loads:
  - current panel live status

    - time-range telemetry (`1H`, `24H`, `7D`)
    - panel metadata
- Features:
    - instant KPI strip
    - anomaly insights (voltage, PF, frequency thresholds)
    - historical charts
    - relay command dispatch (ON/OFF)
    - RTC schedule synchronization
    - map location card

4.4.6 Manage Panel (`/manage-panel`)

- Create new panel or edit existing panel metadata.
- Supports geolocation and map-based coordinate picking.
- Enforces admin key for writes (prompts if absent in session).
- Uses POST for create, PATCH for update.

4.4.7 Analytics (`/analytics`)

- Historical telemetry query across one or more panels.
- Date-range query form.
- Visualization modes:
    - Overview charts by metric
    - Multi-metric compare across selected nodes
    - Raw telemetry table
    - Custom chart builder (line/area/bar)
- CSV export of queried telemetry.

4.4.8 Alerts (`/alerts`)

- Displays active/acknowledged alerts.
- Severity and status filters.
- Acknowledge action updates client-side alert state.

4.5 Frontend API Integration Design

`ccms/lib/api/http.ts` centralizes HTTP behavior:

- Reads `NEXT_PUBLIC_API_BASE_URL`

- Adds authentication headers from session store
- JSON serialization/parsing
- Unified error throw for non-2xx responses

`ccms/lib/api/ccms-api.ts` provides domain functions:

- Converts Lambda raw snapshot/history data into typed UI structures.
- Adds UI-level derived metrics and defaults.
- Isolates route components from backend response shape changes.

4.6 Data Contracts and Type System

`ccms/lib/api/types.ts` defines stable contracts used by UI screens:

- `DashboardSummary`
- `PanelRecord`, `PanelLiveStatus`
- `TelemetryPoint`, `TelemetryResponse`
- `AlertRecord`, `AlertListResponse`
- `PanelCommandPayload`

This helps avoid ad-hoc parsing in components and improves maintainability.

4.7 UI and UX Characteristics

- Dark industrial theme aligned with operations center use case.
- Shared UI primitives for consistency (headers, cards, banners, chips).
- Map interactions for spatial awareness.
- Soft real-time behavior through periodic refresh and user-triggered refresh.

4.8 Frontend Deployment Model

Because `next.config.ts` sets `output: "export"`:

- App can be hosted as static files (CDN/object storage/GitHub Pages-like setups).
- No Next.js server runtime required.
- API base URL must be externally reachable and configured via env var.

5. Lambda Functions Detailed Report

5.1 Lambda Inventory

1. `Lambda/lambdaAPI.py`
    - HTTP API handler for metadata, snapshot, history, and mutations.
2. `Lambda/saveDataFromSQStoDB,py`
    - SQS batch ingestion Lambda writing telemetry records to DynamoDB.

5.2 API Lambda (`lambdaAPI.py`) Behavior

5.2.1 Initialization

- Creates clients/resources:
    - SSM client
    - DynamoDB resource
    - `MeterTelemetry` table object
    - `PanelMetadata` table object
- Configures root logger at INFO level.

5.2.2 Security Model

Per-request key validation using SSM parameters:

- `/api/dashboard_key` (Level 1)
- `/api/admin_key` (Level 2)

Rules:

- All requests require valid dashboard key.
- Mutating methods (`POST`, `PUT`, `PATCH`, `DELETE`) require valid admin key.

5.2.3 CORS

Always returns permissive CORS headers:

- `Access-Control-Allow-Origin: *`
- Allowed headers include dashboard/admin keys.

`OPTIONS` returns `200` for browser preflight.

5.2.4 Endpoint Logic by Method

`GET`

- `?enquiry=history`
  - Validates panel id and timestamp range.
  - Queries `MeterTelemetry` by `meter_id` and timestamp range.
  - Returns ordered time-series points.
- `?enquiry=snapshot`
  - Scans all `PanelMetadata` records.
  - For each panel, queries latest 5 telemetry records.
  - Returns array of `{ metadata, recent_logs }`.
- default metadata read
  - Requires `panel_id` in query or body.
  - Performs `get_item` on `PanelMetadata`.

`POST` / `PUT`

- Requires body with `panel_id`.
- Writes full item into `PanelMetadata` (`put_item`).

`PATCH`

- Requires target `panel_id` and at least one update field.
- Dynamically builds update expression and patches selected fields.

`DELETE`

- Validates existence and deletes panel metadata entry.

5.2.5 Validation and Error Handling

- JSON body parse guard with 400 on malformed JSON.
- Input validation helpers for panel id and timestamps.
- Maps DynamoDB errors to user-facing HTTP codes/messages where possible.
- Top-level exception fallback returns 500 with generic message.

5.2.6 JSON Serialization

Uses custom `DecimalEncoder` to convert DynamoDB Decimal values before response serialization.

5.3 Ingestion Lambda (`saveDataFromSQStoDB.py`) Behavior

5.3.1 Trigger and Batch Handling

- Triggered by SQS event with `Records` batch.
- Iterates each message independently.
- Returns `batchItemFailures` for partial retry semantics.

5.3.2 Record Processing

For each message:

1. Parse JSON payload with `parse_float=Decimal`
2. Resolve partition key:
    - `meter_id = payload.client_id` (fallback `UNKNOWN_METER`)
3. Resolve timestamp sort key:
    - `server_time` preferred, else `timestamp`, cast to int
4. Copy remaining telemetry fields into DynamoDB item
5. `put_item` into `MeterTelemetry`

5.3.3 Failure Behavior

- Any record-level exception logs failure and appends message id to batch failures.
- Failed messages remain in queue for retry.

5.4 Operational Characteristics of Lambda Layer

Strengths:

- Clear separation between API and ingestion responsibilities.
- Admin/user authorization split.

- Explicit validation path for critical query params.
- Partial batch retry handling in ingestion path.

Constraints:

- Snapshot uses table scan + per-panel query, which can become expensive at scale.
- Key-based auth is shared-secret style and lacks rotation/session semantics in API design.

6. Frontend <-> Lambda Integration Contracts

6.1 Required Environment

Frontend requires:

- `NEXT_PUBLIC_API_BASE_URL`

Runtime behavior assumes endpoint paths like:

- `DashboardAPIHandler?enquiry=snapshot`
- `DashboardAPIHandler?enquiry=history&panel_id=...`

6.2 Header Contract

- Required for all API calls: `x-dashboard-key`
- Required for mutations: `x-admin-key`

6.3 Core Functional Mappings

- Dashboard, panels, alerts all derive from `enquiry=snapshot`.
- Panel command writes call API `PATCH` with desired state/schedule fields.
- Analytics/history calls use `enquiry=history` + timestamp range.

8. Testing and Verification

8.1 Existing Test Artifacts

- `Lambda/TestEndpoints/getData.js` demonstrates API usage patterns.
- Frontend:
- Unit tests for API mapper functions (`ccms-api.ts`)
- Route-level integration tests for auth redirects and mutation flows
- E2E tests for login -> dashboard -> panel command paths

Lambda API:

- Contract tests by method and auth level
- Validation tests for timestamp and panel_id edge cases
- Error mapping tests for DynamoDB and SSM failures

Ingestion Lambda:

- Batch tests with mixed good/bad records
- Type consistency tests for timestamp and meter id
- Idempotency/duplication handling checks

9. Deployment and Operations Notes

Frontend:

- Build: `npm run build` in `ccms/`
- Host generated static output on CDN/static hosting
- Ensure API URL and CORS are configured correctly

Lambda:

- Confirm SSM parameters exist (`/api/dashboard_key`, `/api/admin_key`)
- Confirm table names match deployed DynamoDB resources
- Verify IAM permissions for:
  - SSM `GetParameters`
  - DynamoDB read/write on both tables